

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Dev Commands

Backend (from repo root):

```
uvicorn main:app --reload --port 8000
```

Frontend (from `frontend/`):

```
npm run dev      # Vite dev server on :5173, proxies /api → localhost:8000
npm run build    # Outputs to frontend/dist/ (served by FastAPI in prod)
npm run lint
```

Both servers must run simultaneously in development. The FastAPI app also serves `frontend/dist/` as the SPA in production via a catch-all route.

Core Principles

Security and schema integrity first. These are non-negotiable constraints, not trade-offs. Every decision about data access, schema shape, and inter-org data flow must pass a security and integrity check before anything else.

Named schema fields are reserved for universal production truths. Promoted fields on canonical tables (e.g. `canonical_assets`, `replicated_assets`) must represent facts that are universally true across all studios and source tools: asset name, product link, project date, asset number. Studio-specific or source-specific data belongs in `meta` (the JSONB payload), not in named columns. Before nominating any field for promotion to a named schema slot, explicitly flag it for review. The bar is high: if it only applies to some studios, or if it duplicates what `meta` already carries, it stays in `meta`.

Client data is unreleased IP. Treat it accordingly. Studios trust ArHound with pre-release game assets, schedules, and production plans. Data must never cross org boundaries without explicit authorization (RLS, dispatch tokens, or direct studio action). No cross-tenant queries, no leaking of studio data to vendors beyond what was explicitly dispatched, no logging of payload content at levels visible outside the system.

PAW: Product, Asset, Work

PAW is the conceptual spine of ArHound. Every piece of data in the system belongs to one of these three tiers:

- **Product:** the project or production context (a game title, a film, a season)

- **Asset:** the discrete creative unit (a character, prop, environment, VFX element)
- **Work:** the task or deliverable attached to an asset (a modelling pass, a review cycle, a vendor delivery)

All three tiers will eventually have their own named schema fields and studio-specific meta. Each studio defines their own $P \rightarrow A \rightarrow W$ hierarchy via `source_entity_definitions`, and connectors translate source tool data into this unified shape.

Asset is the most important entity in the system. The canonical asset is the stable, cross-tool identity that everything else anchors to. It is the thread that connects a studio's internal production data, vendor deliveries, reviews, schedules, and generated work into a single coherent record of that asset's journey through development.

Everything must link to its canonical asset. No exceptions. Work items, dispatches, asset reviews (studio and vendor side, regardless of collaboration model), source keys, change records, vendor deliveries, ingested records, none of these have meaning without their canonical asset link. An unlinked record is an orphan and has no place in the system. When any operation would produce a record without a resolved canonical asset link, abort rather than persist the incomplete state.

Connectors exist to translate source tool idioms (Jira epics/stories, Airtable linked records, ShotGrid tasks) into the PAW shape. The connector's job is to make the source tool's structure conform to ArtHound's model, not the other way around.

Architecture

ArtHound is a **canonical production data layer** for game studios. It replicates source tool data (Airtable, Jira, and future connectors) into Supabase, then all product features read from Supabase, never from the source tool directly. This is the single most important architectural principle.



Sync is triggered on login (via `AuthContext.jsx`) and by a background polling loop in `main.py`. The webhook handler in `routes/sync.py` will also trigger it. All three paths call the same sync function, keep sync logic trigger-agnostic.

Backend Conventions

Auth: Every protected route uses `get_current_user` from `lib/auth.py` as a FastAPI dependency. It verifies the Supabase JWT (JWKS + HS256 fallback), resolves the user's role (`studio` or `vendor`), and returns membership info with a 15-second cache. Do not re-implement auth logic inline.

Supabase queries: Use the helper in `lib/db.py` which builds the PostgREST URL and injects service-role headers. All DB writes use the service role key, not the anon key.

Airtable calls: `lib/airtable.py` has been deleted. All Airtable access goes through `lib/sync/connectors/airtable.py` (AirtableConnector). New features must read from Supabase replicated tables, not call Airtable directly.

Credentials: Source credentials (Airtable PAT, etc.) are encrypted at rest in `source_credentials`. Use `lib/source_creds.py` to retrieve and decrypt them; never query that table directly in route handlers.

Field mapping: Source fields are mapped to ArtHound slots via `source_field_mappings` in Supabase. Estimation and task queries must resolve field names through this table, never assume a column name. See `lib/sync/normalizer.py` for how slots are resolved.

[IGNORE] field convention: Any source field whose name begins with `[IGNORE]` (case-insensitive) is auto-suppressed at classification time, `ingest_suppressed=True`, `display_tier="hidden"`, excluded from `meta`. Studios use this prefix to mark internal plumbing fields (link-back columns, formula sources) that should never appear in the ArtHound UI or be ingested into replicated records.

MCP Server (external agent access)

`mcp_server/` exposes ArtHound data to external AI agents over the Model Context Protocol, mounted at `/mcp` in `main.py` (FastMCP, Streamable HTTP). Agents authenticate with an API key (`scripts/agent_keys.py`) that resolves to a `member_role='agent'` service-account member of one org (identity model A1) — so **all tool DB access runs under the agent's bound JWT and the existing RLS, never service-role**. Within-org scope (read/write, tool allowlist) is enforced in `mcp_server/context.py:tool_call`. Tools are versioned `paw_v1_*` and PAW-shaped; writes carry mandatory `actor_type` / `actor_ref` and abort rather than persist an orphan. Agent-written records (flags/review-requests/estimate-proposals) surface in the UI via `/api/agent-activity` (Asset viewer **Agent** tab + home widget). Full reference: [docs/mcp.md](#); design + phases: [docs/plans/mcp-server.md](#). Do not add a `raw_query` /SQL tool or service-role access to this surface.

Frontend Conventions

Component library: All UI is built from the primitives in `frontend/src/components/ui/` (import via the barrel: `Button`, `Input`, `Select`, `Textarea`, `Field`, `Modal`, `Pill`, `StatusDot`, `Tabs`, `Card`, `KV`, `Table` / `Th` / `Tr` / `Td`, `EmptyState`, `Spinner`, `Skeleton`, `Dropdown`, `PageHeader`, `SectionLabel`). Never hand-roll buttons, inputs, modal overlays, badges, or table styling in feature code. Full usage rules live in `docs/ui-redesign/MIGRATION.md`. Key invariants: - Status/priority/craft colors resolve

through `frontend/src/lib/statusColors.js` — no hardcoded hex in JSX. Statuses inside data tables render as `<StatusDot>` (dot + plain text), not pills; `<Pill>` is for role/sync/dispatch badges. - Border tiers: `border-border` (structural), `border-border-soft` (internal dividers, table rows), `border-border-faint` (hairlines). No ad-hoc `border-border/NN` opacities. - Text tiers: `text-foreground` / `text-muted` / `text-faint`. Links are `text-link hover:underline`. - Radii: `rounded-md` (6px) controls, `rounded-lg` (8px) panels/cards/modals. Data tables are fixed-density (30px rows via the `Table` primitives). - Theme tokens live in `frontend/src/index.css` (`@theme`); the chrome is neutral graphite — the lavender accent is reserved for actions, selection, and focus.

API calls: Always use `apiFetch()` from `frontend/src/lib/api.js`. It injects the Supabase JWT and standardizes error handling. Never use raw `fetch` for `/api` routes.

Auth state: Supabase session and user profile live in `AuthContext`. App-level UI state (open modals, selected assets, filters) lives in `AppContext`. Hooks in `frontend/src/hooks/` encapsulate all data fetching.

Props → state sync: When a component initializes `useState` from props, add a `useEffect` to sync if the prop changes (e.g., `useEffect(() => setState(prop), [prop])`). Components that open from a list and show `history[0]` are the main case.

Field schema: When passing field definitions between layers, always include the full type + options object (select choices, `LinkedTableId`, formula result type). Do not strip to just the type name, downstream rendering depends on options.

Page centering: Page content must always be horizontally centered — it must never hug the left edge with dead space on the right. Standard content pages render their body inside `PageContainer` (`frontend/src/components/PageContainer.jsx`), which bakes in `mx-auto` + a max-width column so centering can't be forgotten; pick the column via the `width` prop (`sm` / `md` / `lg`) and keep per-page padding/gap in `className`. Never put a bare `max-w-*` on a page's root `<main>` without `mx-auto` (that is the exact left-align bug `PageContainer` exists to prevent). Pages rendered as panels inside another layout (e.g. `OrgHub` tabs) center via `max-w-* mx-auto w-full` instead. The only exception is intentionally full-bleed multi-panel tools — `Assets` (3-panel), `Reviews` (split), `Estimates / EstimatesTab` (matrix grid), `ScenarioPlanner` (chat) — which fill the viewport by design and manage their own layout.

Database

Migrations live in `supabase/migrations/` and are applied in filename order. All schema changes must go through migration files, never via the Supabase dashboard.

Identity & tenancy: - `studios`, `vendors`, org records - `studio_members`, `vendor_members`, user → org membership - `studio_vendor_links`, `studio_vendor_invites`, org-level studio↔vendor connections - `canonical_assets`, global stable asset IDs; keyed on `(studio_id, source_record_id, source_type)`

Sync layer: - `replicated_assets` / `replicated_products` / `replicated_item_types` / `replicated_work`, synced source data; all features read from here - `replicated_assets.meta["__slots"]`, ArtHound-normalized values for demoted slots (`item_type`, `status`, `priority`, `team`, `dev_name`). Read via `meta.get("__slots", {}).get(slot)`. Never render `__slots` as a raw meta field, it is excluded from `_META_HIDDEN` in `routes/assets.py` and must stay excluded. - `source_field_mappings`, per-studio field → ArtHound slot mapping - `source_entity_definitions`, P→A→W hierarchy per studio/vendor (which source table is Products, Assets, Work; linking fields; filters) - `source_credentials`, encrypted source tokens (service role only; use `lib/source_creds.py`) - `sync_cursors`, `sync_log`, sync state and audit trail

Work generation: - `generated_work`, ArtHound-generated work snapshots; always filter `deleted_at IS NULL` unless querying history - `workflow_steps`, `workflow_step_dependencies`, workflow definitions. Org-scoped (dual FK `studio_id` / `vendor_id` + generated `owner_key`). Dependency cycles rejected server-side in `routes/workflow_steps.py` via `lib/workflow_graph.py` - `estimate_matrix`, `estimate_config`, estimation system. Org-scoped via `owner_key`. `estimate_matrix.link_id` → `studio_vendor_links` carries vendor per-link rate overrides (NULL = base row)

Vendor estimate sharing (vendors share frozen rate snapshots with linked studios; reverse of payload dispatch, see `docs/estimate-sharing.md`): - `estimate_share_series`, one sharing channel per (`vendor_id`, `link_id`) - `estimate_share_dispatches`, frozen `snapshot` JSONB at a chosen granularity; replace semantics (`superseded_at`), `revoked_at` / `expires_at`; one-live-per-channel partial unique. Deliberately NOT FK'd to `canonical_assets` (rate card is a tier above any asset) - `estimate_share_access_log`, append-only (`shared` / `viewed` / `revoked` / `superseded`); service-role writes only

Payload / vendor dispatch: - `payload_dispatches`, studio-to-vendor asset payloads (token, expiry, revoke state) - `payload_field_mappings`, vendor's saved field mapping + ingest state (`ingested_at`, `failed_at`, `failure_reason`, `ingested_source_record_id`) - `payload_templates`, vendor's saved default mapping per studio link - `payload_export_records`, canonical link between vendor's created source record and the canonical asset; written on successful ingest - `failed_ingests`, quarantine for records where external write succeeded but canonical link failed; used by `/retry-canonical` - `vendor_studio_ingest_templates`, vendor ingest template snapshots per studio link

Reviews & attachments (cross-org review system v2, see `docs/reviews.md` + `docs/plans/cross-org-reviews.md`): - `asset_reviews`, ArtHound-native reviews (never synced). `scope` ∈ `internal|cross_org`; cross-org rows carry `link_id`. Internal reviews cross the org wall ONLY via the `promote_review` RPC (trimmed copy, `promoted_from_review_id`). Accepted reviews (`accepted_at` + `frozen_snapshot`) are fully immutable; re-delivery = new review via `revision_of_review_id`. Identity/provenance columns are trigger-guarded immutable - `review_assets`, m2m asset junction (primary `canonical_asset_id` stays NOT NULL on the review and is mirrored in) - `review_comments`, threaded comments with visibility lanes (`internal|shared`, default internal everywhere; one-way flip, trigger-enforced). `frozen_snapshot` must only ever contain shared-lane comments — it is visible to both link parties - `review_events`, append-only audit; NO user INSERT policy — routes write via `system_identity()`, RPCs in-transaction - `review_trim_templates`, vendor-owned promotion trim configs (payload_templates analogue) - `review_workflow_def` / `review_step_def`, org-scoped link

protocols (required submissions; org-scope stack convention). Link FK

`studio_vendor_links.review_protocol_def_id`; steps archive, never delete. The requirements checklist is computed at read (`/api/reviews/requirements`); out-of-protocol step tags are inert - Review RPCs (all `arthound_rpc` -owned DEFINER): `promote_review`, `review_set_status`, `review_set_link_protocol`, `review_accept`. Never overload an RPC signature — PostgreSQL cannot dispatch overloads; DROP the old signature first - `review_attachments`, files attached to reviews. Promoted copies reference the SAME storage path (`copied_from_attachment_id` provenance); blob deletion must first check for other rows referencing the path (done under `system_identity()` in `delete_attachment`) - `studio_vendor_links.review_collaboration_mode` is vestigial — kept but unread - `attachment_copy_jobs`, `attachment_refs`, copy-on-demand attachment pipeline to Supabase Storage

Meta / schema classification: - `field_bucket_override_log`, `schema_drift_events`, meta bucket classification and drift tracking

Agent access (MCP) (see `docs/mcp.md`): - `agent_credentials`, API key (sha256 hash) → org + principal member + `scopes` + expiry/revoke. RLS+FORCE, system-only + `REVOKE` from anon/authenticated (key hashes never reach a user token) - `agent_access_log`, append-only agent identity/tool audit (system-write) - `asset_flags`, `review_requests`, `estimate_adjustment_proposals`, agent-write paper-trail records (own-org RLS via `is_my_org`; `actor_type` / `actor_ref`; read/triaged via `/api/agent-activity`). `estimate_adjustment_proposals.canonical_asset_id` is nullable by design (estimate tier is above any asset — estimate_share carve-out) - `studio_members` / `vendor_members.member_role` CHECK includes `'agent'` (the non-human service-account seat)

Multi-tenancy

Studios and vendors are separate roles with separate home pages (`StudioHome.jsx` / `VendorHome.jsx`). Field mappings, source credentials, and sync cursors are all scoped to `studio_id` or `vendor_id`. The `source_entity_definitions` table defines each org's P→A→W hierarchy, nothing about the source table structure should be assumed or hardcoded. Data never crosses org boundaries without explicit authorization: RLS policies, dispatch tokens, or a direct studio action. Vendor data is scoped to the vendor; studio data is scoped to the studio; shared data (dispatches, reviews) requires an explicit link between the two.

Known Debt

- **Jira write-back edge cases** (`routes/schedule.py`): Sub-task creation (requires `parent.key`), missing `_jira_key` warning not surfaced to UI, per-item failure detail not returned to frontend.
- **sync_log retention**: Nightly trim keeps 100 rows/owner (`SYNC_LOG_KEEP_ROWS` , default 100). `trim_sync_log()` Postgres function, migration `20260511000003`. Skips `running` rows.
- **Products and item types have no field mapping** (`lib/sync/normalizer.py`, `lib/sync/runner.py`): `source_field_mappings` only covers the asset entity. Products and item

types are normalized via `normalize_reference()` with a primary-field heuristic: the runner fetches the schema for each entity's own table and uses its first field as the name key (separate-table setups), or falls back to the asset name-slot field for flat-table setups. No slot mapping beyond name, product fields like status, owner, and deadline are never promoted to named slots. Proper fix is part of the full PAW product field schema design.